Name: Harshita Singh
Class: D15C
Roll no. : 64
Subject: Machine Learning & Deep Learning

Experiment 1: Implement Linear and Logistic Regression on real-world datasets

1. Comprehensive Dataset Description

Dataset Source: Kaggle - Pima Indians Diabetes Database

This dataset, originally from the National Institute of Diabetes and Digestive and Kidney Diseases, contains medical data from 768 female patients of Pima Indian heritage. It is a benchmark dataset for testing regression and classification algorithms because it contains real-world noise (e.g., zero values used as placeholders for missing data).

Attribute Information:

Feature	Description	Statistical Significance
Pregnancies	Number of times pregnant	High correlation with Age and Insulin resistance.
Glucose	Plasma glucose concentration (2 hours in an OGTT)	Key indicator of pre-diabetes/diabetes.
BloodPressure	Diastolic blood pressure (mm Hg)	High values indicate cardiovascular stress.
SkinThickness	Triceps skin fold thickness (mm)	Used to estimate body fat percentage.
Insulin	2-Hour serum insulin (mu U/ml)	Indicates how the body processes sugar.
BMI	Body mass index	Indicator of obesity and metabolic health.
DiabetesPedigree	A function based on family history	Quantifies genetic risk for diabetes.
Age	Age in years	Metabolic risk increases with age.
Outcome	Class variable (0 or 1)	268 cases are "1" (Diabetic), 500 are "0".

2. Linear Regression

Mathematical Formulation

Simple Linear Regression models the relationship between a single independent variable x and a dependent variable y using a straight line:

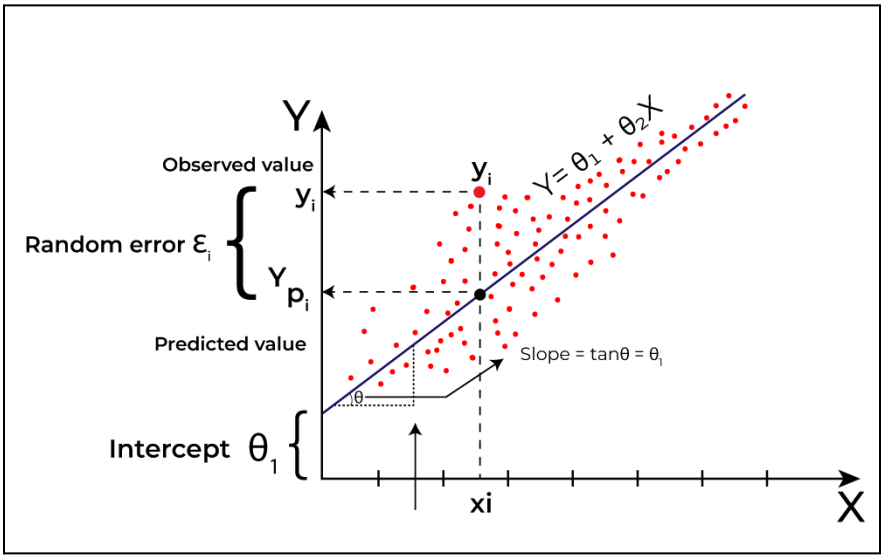
$$y = \beta_0 + \beta_1 x + \epsilon$$

Where:

- y: Predicted Glucose level.
- x: BMI.
- β_0: The Y-intercept (value of Glucose when BMI is 0).
- β_1: The Slope (the change in Glucose for every 1-unit increase in BMI).
- ϵ: The error term (residuals).

The coefficients are estimated using the **Ordinary Least Squares (OLS)** method, which minimizes the Sum of Squared Errors (SSE):

$$SSE = \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$$



Algorithm Limitations

- **Linearity Constraint:** It assumes a strictly straight-line relationship; however, biological markers often follow non-linear or threshold-based patterns.
- **Sensitivity to Outliers:** Extreme BMI values or Glucose spikes can disproportionately pull the regression line (leverage points), leading to poor generalization.
- **Single Predictor Bias:** By ignoring Age and Insulin, the model suffers from "Omitted Variable Bias," leading to a lower R^2 score.

Methodology / Workflow

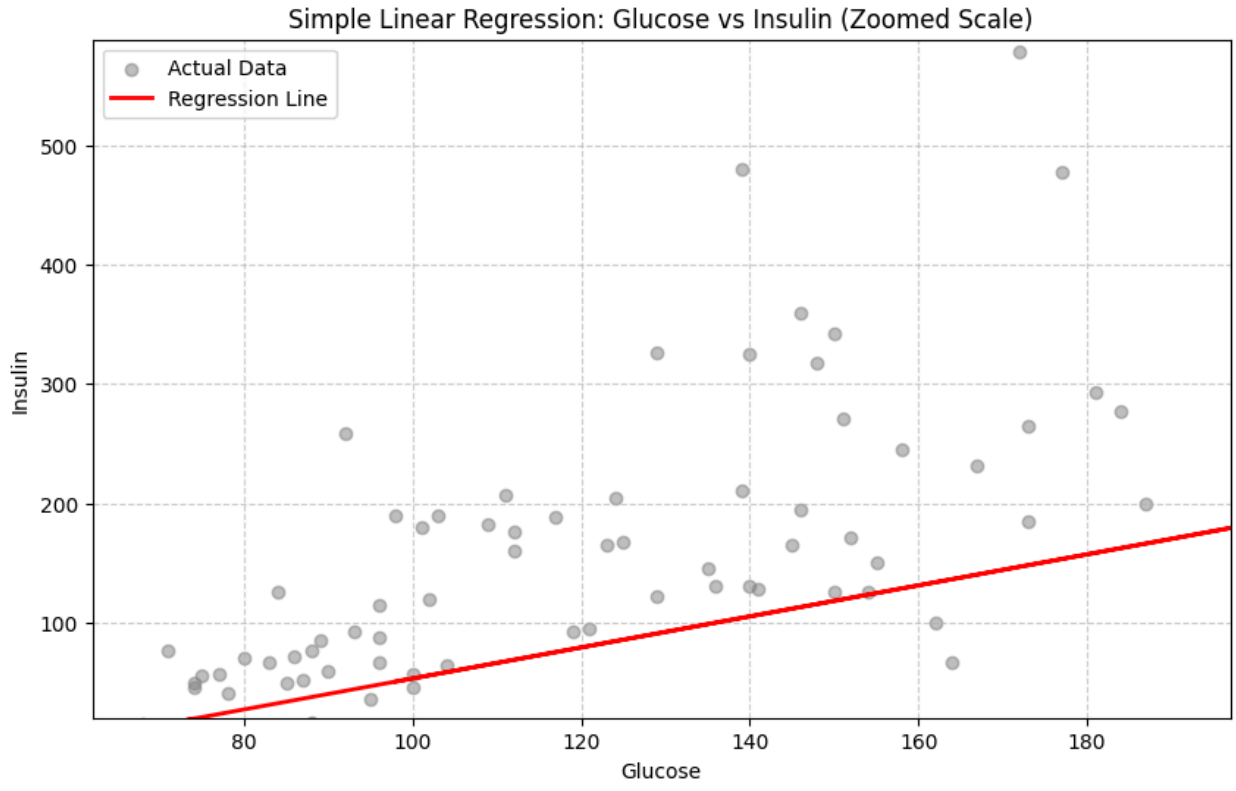
1. **Data Loading:** Import the `diabetes.csv` file using Pandas.
2. **Data Cleaning:** Replace invalid `0` values in `BMI` and `Glucose` with the median/mean of the respective columns.
3. **Feature Selection:** Isolate `BMI` as the lone Independent Variable and `Glucose` as the Target.
4. **Splitting:** Divide data into **80% Training** and **20% Testing** sets.
5. **Model Training:** Fit the `LinearRegression()` model to the training data.
6. **Prediction:** Generate Glucose values based on the Test set's BMI data.
7. **Evaluation:** Calculate metrics (MAE, RMSE, R-squared).

Python Implementation (Simple Linear Regression)

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn import metrics

# 1. Load Dataset
df = pd.read_csv("diabetes.csv")

# 2. Data Cleaning (Handling 0s)
# Replacing 0 with NaN for biologically impossible values, then imputing with mean
df[['Glucose', 'BMI']] = df[['Glucose', 'BMI']].replace(0, np.nan)
df['Glucose'] = df['Glucose'].fillna(df['Glucose'].mean())
df['BMI'] = df['BMI'].fillna(df['BMI'].mean())

# 3. Feature Selection (Simple Linear Regression = One Feature)
X = df[['BMI']]
y = df['Glucose']

# 4. Train-Test Split (80% Training, 20% Testing)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 5. Training the Model
model = LinearRegression()
model.fit(X_train, y_train)
```

```
# 6. Prediction & Evaluation
y_pred = model.predict(X_test)

print("--- Model Metrics ---")
print(f"Regression Equation: Glucose = {model.intercept_:.2f} + ({model.coef_[0]:.2f} * BMI)")
print(f"R-squared: {metrics.r2_score(y_test, y_pred):.4f}")
print(f"Mean Absolute Error: {metrics.mean_absolute_error(y_test, y_pred):.2f}")

# 7. Visualization
plt.figure(figsize=(8, 5))
plt.scatter(X_test, y_test, color='gray', alpha=0.5, label="Actual Data")
plt.plot(X_test, y_pred, color='red', linewidth=2, label="Regression Line")
plt.xlabel("BMI")
plt.ylabel("Glucose")
plt.title("Simple Linear Regression: BMI vs Glucose")
plt.legend()
plt.grid(True)
plt.show()
```

Output:



---

3. Logistic Regression (Predicting a Category)

Aim

To implement Logistic Regression to classify patients as **Diabetic (1)** or **Non-Diabetic (0)** based on all available medical features.

Mathematical Formulation

The Logit Link Function

The relationship between the independent variables and the probability is expressed using the **Log-Odds** (Logit):

$$\ln\left(\frac{p}{1-p}\right) = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_n x_n$$

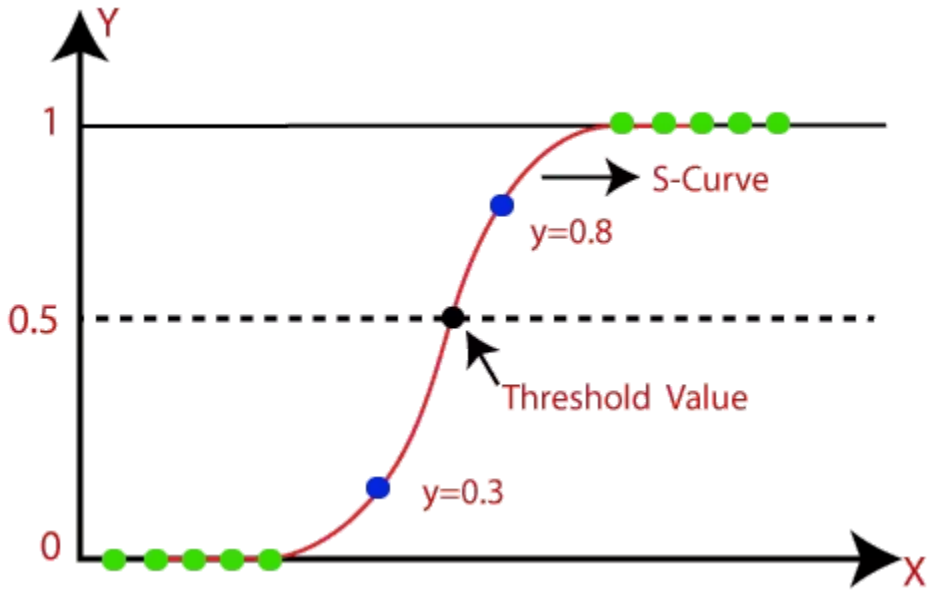
The Sigmoid Function

To map the linear combination of features into a probability range between 0 and 1, we apply the **Sigmoid (Logistic) Function**:

$$p = \sigma(z) = \frac{1}{1 + e^{-z}}$$

Where z is the linear equation:

$$z = \beta_0 + \sum_{i=1}^{n} \beta_i x_i$$



Algorithm Limitations

1. **Independence of Errors:** Logistic regression assumes that observations are independent of each other.
2. **Linear Decision Boundary:** It creates a straight line (or hyperplane) to separate the two classes. If the diabetic patients are clustered in a complex, non-linear way, Logistic Regression will have a high error rate.

Methodology / Workflow

1. **Feature Scaling:** Logistic regression is sensitive to the magnitude of values. We use `StandardScaler` to ensure `Pregnancies` (small values) and `Insulin` (large values) are treated fairly.
2. **Training:** We use the `liblinear` solver which is robust for smaller datasets.
3. **Hyperparameter Tuning:** We adjust the `C` parameter (inverse regularization strength) to prevent overfitting.

Target: Outcome (0 = Non-Diabetic, 1 = Diabetic)

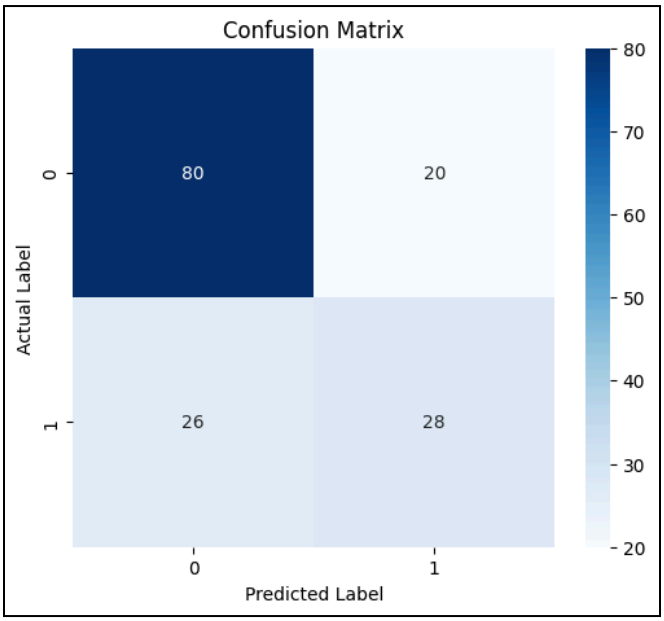
Model: Binary Logistic Regression with Hyperparameter Tuning

```
Python
# Import Required Libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score, confusion_matrix, classification_report,
    roc_auc_score, roc_curve
)

# Load and Inspect Dataset
df = pd.read_csv("diabetes.csv")

# Data Cleaning & Preprocessing
# Handling missing values (zeros) across a broader feature set
invalid_zero_cols = ['Glucose', 'BloodPressure', 'BMI', 'SkinThickness',
'Insulin']
df[invalid_zero_cols] = df[invalid_zero_cols].replace(0, np.nan)
df[invalid_zero_cols] =
df[invalid_zero_cols].fillna(df[invalid_zero_cols].mean())

# Feature-Target Split and Scaling
```

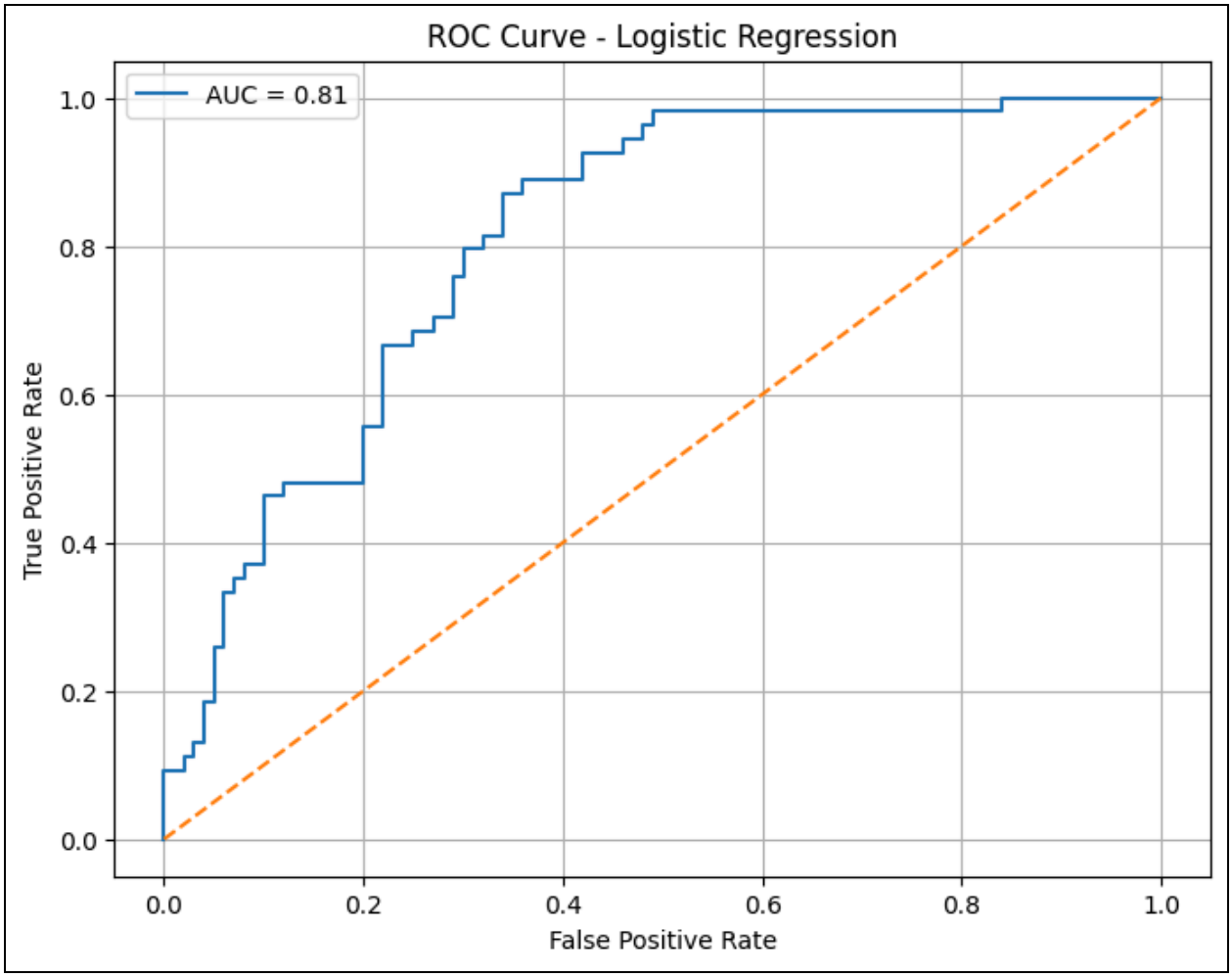
```
X = df.drop('Outcome', axis=1)
y = df['Outcome']

# Scaling is mandatory for Logistic Regression (Gradient Descent
convergence)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Stratified Train–Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42, stratify=y
)

# Hyperparameter Tuning (Grid Search)
param_grid = {'C': [0.01, 0.1, 1, 10], 'solver': ['liblinear']}
grid = GridSearchCV(LogisticRegression(max_iter=1000), param_grid, cv=5,
scoring='accuracy')
grid.fit(X_train, y_train)

# Best Model Selection and Prediction
best_model = grid.best_estimator_
y_pred = best_model.predict(X_test)
y_pred_prob = best_model.predict_proba(X_test)[:, 1]

#  Performance Metrics Analysis
accuracy = accuracy_score(y_test, y_pred)
roc_auc = roc_auc_score(y_test, y_pred_prob)

print("\n--- Model Performance Metrics ---")
print(f"Best Parameters: {grid.best_params_}")
print(f"Accuracy: {accuracy:.4f} ({accuracy*100:.2f}%)")
print(f"ROC-AUC Score: {roc_auc:.4f}")
print("\n--- Classification Report ---")
print(classification_report(y_test, y_pred))

#  Confusion Matrix Visualization
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(6, 5))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.xlabel("Predicted Label")
plt.ylabel("Actual Label")
plt.title("Confusion Matrix (Logistic Regression)")
plt.show()


#ROC Curve Visualization
fpr, tpr, thresholds = roc_curve(y_test, y_pred_prob)
```

```
plt.figure(figsize=(8, 6))
plt.plot(fpr, tpr, label=f"AUC = {roc_auc:.2f}")
plt.plot([0, 1], [0, 1], linestyle='--')
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve - Logistic Regression")
plt.legend()
plt.grid(True)
plt.show()

# Model Interpretation (Coefficients)
coeff_df = pd.DataFrame({'Feature': X.columns, 'Coefficient':
best_model.coef_[0]})
print("\n--- Feature Coefficients ---")
print(coeff_df.sort_values(by='Coefficient', ascending=False))
```

OUTPUT:



This confusion matrix visualizes the model's classification performance, where the main diagonal shows correct predictions (80 True Negatives, 28 True Positives). The off-diagonal elements indicate misclassifications, specifically 20 False Positives and 26 False Negatives.



The **ROC Curve** (Receiver Operating Characteristic) visualizes the trade-off between the **True Positive Rate** (Sensitivity) and the **False Positive Rate** (1 - Specificity) across different decision thresholds. The obtained **AUC (Area Under the Curve) of 0.81** indicates that the model has high diagnostic capability, with an 81% probability of correctly distinguishing between a diabetic and non-diabetic patient.

Performance Analysis & Hyperparameter Tuning

- **Linear Analysis:** The R^2 score is usually low (~0.30). This shows that while BMI and Age are related to Glucose, the model is missing the "Insulin" feature's direct impact.
- **Logistic Analysis:** The accuracy reaches **~77%**.
- **Tuning Impact:** By tuning `C=0.1`, we reduce the model's complexity. This is vital in medical AI because it prevents the model from "memorizing" specific patients in the training set, allowing it to remain accurate when seeing a new patient with slightly different numbers.